# Keyfinix Hierarchical Authentication Element Encoder Scheme

**Yumechi yumechi@yumechi.jp**

## Contents

# Construction

For:

- a secret key $k$, and SipHash128 PRF $S_h$, and $S_l$ returns the 64-bit halves of the output of SipHash.
- two functions $t_0(d)$ and $t_1(d)$, returns a 64-bit pseudo-random value type ID for the $d$-th auth element in the hierarchy that is distinct for each element but not secret.
- a function $m(d)$ that returns an arbitrary, potentially attacker controlled string for the row level data for the $d$-th auth element.

We define recursively the semantic authentication data $m(d)$ for the tuple of functions $< t_0(d), t_1(d), m(d) >$:

$$m(0) = 1 \parallel (t_1(0) \oplus (S_l(1) \lll 17)) \parallel m(0) \parallel ((t_1(0) + 1) \oplus S_h(1))$$

$$m(d) = m(d-1) \parallel d \parallel (t_1(d) \oplus (S_l(m(d-1) \parallel d) \lll (17 * (d+1)))) \parallel m(d) \parallel ((t_1(d) + d) \oplus S_h(m(d-1) \parallel d))$$

The semantic collision resistant authentication material produced by the encoder is $S_h(m_d) \parallel S_l(m_d)$

The desired property is that, at any arbitrary round $r$, the attacker cannot find a specific $m'$ such that $m_d$ equals another $m'_{d'}$ where at least one of $d$, $m(d') \mid d' \neq r$, $t_0(d')$ or $t_1(d')$ is different.

# brainstorm

## Why not Double Hash?

A very intuitive solution one might believe to be "better" is why do you reuse the same hasher for each element but not double hash? Just create a new hasher, key it, pass it to the row level encoder, get the hash, then double hash into the actual encoder.

This has two huge footguns:

You now have a KDF problem here, how do you ensure you don't just hand out freshly keyed SipHash registers into a potentially broken implementation? So you must use a different key, how do you do that securely? there are no high performance solutions to that.

## Why not Generic Chaining?

The Vulnerability in Generic Chaining:

In these generic chaining approaches, if a developer writing a sequence of elements to bind makes a mistake, they can endanger the entire chain by:

Length Extension: If they don't bind their internal length correctly, or use a hash vulnerable to length extension, an attacker might be able to append data after their element and compute a valid hash for the extended chain, even without the key.

Splicing/Tampering: If they produce an internal state that's easily reproducible by an attacker for a different set of inputs, or if their serialization leads to ambiguous interpretations, an attacker might be able to splice in a malicious ElementN' that appears valid to the overall chain hash.

You cannot securely "squeeze" an MD construction hash like SHA-2, period. So you need to run at least a new full block of data to "flush" the state into divergent states, which is expensive, this can both create doubts and disincentivize the pervasive use of authentication, or create performance barriers in high throughput use cases like large scale rekeying.

## Feed-forward construction

the logic is mostly that double-feedforward step cheaply binds the exact structure of the hierarchy thanks to the sponge property of SipHash, and withholding $m(r-1)$ from the row-level constructor for $m(r)$ ensures even with a pre-image attack where one can create a malicious $m'(r)$ on the fly, we still have at least 64-bit of unknown data that an attacker cannot counter immediately mixed in and ultimately diffused by the final suffix term.

In contrast this double feedforward approach first binds that the register the inner implementation sees is already highly dependent on the prefix (and thus cannot be repurposed), and then we mix in that feedforward to make sure no matter what happens, the output is STILL dependent on the prefix, and also dependent on a 64 bit value the attacker do not know

## Security Boundaries

This has nothing to do with the CIA of the cipher-text itself, but for binding the context of a cipher-text to a particular element (for example, the password reset token for a particular user, in the API handler scope).

Thus, the auth data is piped directly into an AEAD cipher, and remote actors generally have minimal saying on the input. Even if a collision was found, the only thing that truly degrades to “plain at-rest encryption” is uniqueness of per-row AD, and this “convolutionary” construction makes the mathematical operation for different length auth chains to be distinct, a typical academic collision attack would likely not have helped in meaningfully causing collision in associated data.